

UNIVERSITÀ POLITECNICA DELLE MARCHE
INGEGNERIA INFORMATICA E DELL'AUTOMAZIONE

Controllo di trazione e sterzata di un veicolo in scala 1:10 mediante radiocomando



Corso di
LABORATORIO DI AUTOMAZIONE
Anno accademico 2025-2026

Studenti:

Tarek Naja
Marco Sambughi
Sara Vaccaro

Professore:

Andrea Bonci

Dottorando:

Alessandro Di Biase



Dipartimento di Ingegneria dell'Informazione

Indice

Introduzione	2
1 Descrizione del sistema	4
1.1 Controllo veicolo	5
1.1.1 Controllo della trazione	5
1.1.2 Controllo dello sterzo	7
2 Hardware	8
2.1 Telaio e meccaniche di supporto	8
2.2 Nucleo-144 STM32H755zi-q	9
2.3 Motore DC Reely 550er	9
2.4 Driver Motore Cytron MD10C R3	10
2.5 Encoder incrementale CUI Devices AMT102	11
2.6 Servomotore Miuzei MS24	11
2.7 Alimentazione	12
2.7.1 Batteria ai polimeri di litio	12
2.8 Inertial Measurement Unit	13
2.9 Radiocomando e Ricevitore Microzone	13
2.9.1 Funzionamento	14
2.9.2 Canali del ricevitore	15
2.10 Schema dei collegamenti	16
3 Software	17
3.0.1 STM32CubeIDE	17
3.0.2 MATLAB	17
3.0.3 Acquisizione dati via Terminale	17
4 Implementazione del firmware	19
4.1 Diagramma di flusso	19
4.2 Implementazione relativa al radiocomando	21
4.2.1 Impostazioni iniziali timers e calcolo preliminare di frequenza e duty	21
4.3 Gestione singoli componenti	29
4.3.1 Gestione componente1	30
4.3.2 Gestione componente2	30
4.4 Funzionamento complessivo	30
5 Test e risultati	31
5.1 Test1	31
5.2 Test2	31
Conclusioni e sviluppi futuri	32

Appendici	32
A Appendice1	33

Introduzione

Il presente elaborato documenta le fasi di progettazione, sviluppo e validazione del firmware di controllo per un veicolo elettrico in scala 1:10. Nello specifico, il lavoro operativo è stato suddiviso in tre task fondamentali: l'interfacciamento e la decodifica dei segnali radio, la regolazione della trazione per la gestione della velocità e il controllo direzionale dello sterzo, questi ultimi implementati mediante algoritmi di controllo PID. Nel primo capitolo, viene fornita una descrizione generale del sistema, definendo il contesto operativo e le logiche alla base del controllo della trazione e dello sterzo. Successivamente, il secondo e il terzo capitolo offrono una panoramica dettagliata dell'architettura hardware e degli ambienti software impiegati, delineando tutti i requisiti strumentali necessari per la riproduzione del setup sperimentale. Al suo interno viene dapprima analizzata la struttura generale del codice, per poi scendere nel dettaglio degli algoritmi sviluppati per l'acquisizione e l'elaborazione dei segnali del radiocomando, illustrando l'impostazione dei timer, il calcolo del Duty Cycle e la gestione degli interrupt. A partire da queste fondamenta, vengono discussi i codici e le librerie specifiche implementate mediante l'utilizzo di regolatori PID, analizzando dapprima l'erogazione della trazione tramite la lettura dell'encoder, e successivamente il controllo direzionale dello sterzo, inclusa la sua necessaria calibrazione meccanica. Per dimostrare la validità del lavoro svolto, ciascuna di queste tre macro-fasi di implementazione software, è corredata da un'analisi dedicata ai test sul campo e ai risultati ottenuti, permettendo di verificare concretamente la prontezza e la stabilità della risposta dinamica del veicolo. Infine, la relazione si conclude con una panoramica sui possibili sviluppi futuri, delineando potenziali miglioramenti e ottimizzazioni per le successive evoluzioni del prototipo.

1 Descrizione del sistema

La seguente relazione si basa sull'analisi dell'architettura di comunicazione via radio tra radiocomando e microcontrollore. Nel dettaglio, il trasmettitore impiega onde elettromagnetiche per l'invio dei pacchetti dati, sfruttando la modulazione di un'onda portante che codifica le informazioni variando parametri dell'onda stessa, quali l'ampiezza, la frequenza o la fase. L'espressione matematica dell'onda portante è definita dalla seguente funzione:

$$s(t) = A \cos(2\pi f_c t + \phi) \quad (1.1)$$

dove:

- A : ampiezza dell'onda;
- f_c : frequenza portante;
- t : tempo;
- ϕ : fase iniziale.

L'azione dell'utente sui comandi del radiocomando genera una sequenza di bit (segnale elettrico), che l'antenna del trasmettitore provvede a convertire in onda elettromagnetica, irradiandola nello spazio a una determinata frequenza f . Il modulo ricevitore, essendo sintonizzato sulla stessa frequenza, intercetta il segnale e lo riconverte nel formato elettrico. Questi dati vengono poi inoltrati al microcontrollore, il quale ha il compito di interpretare le istruzioni e comandare l'hardware per eseguire determinate azioni. Per la traduzione pratica dei comandi, il microcontrollore impiega la modulazione a larghezza di impulso (PWM). Il parametro fondamentale per questa tecnica è il *Duty Cycle*, calcolato come segue:

$$\text{Duty Cycle (\%)} = \frac{T_{on}}{T_{on} + T_{off}} \times 100 \quad (1.2)$$

dove T_{on} è la durata dell'impulso "alto", T_{off} la durata dell'impulso "basso" e la loro somma definisce il periodo totale dell'onda. Per quanto riguarda l'analisi della traiettoria del veicolo, si adotta il modello cinematico della bicicletta, approssimando la dinamica del veicolo a quattro ruote riducendola a due. Le equazioni cinematiche descrivono il moto attraverso le variabili di posizione (x, y) e l'angolo di orientamento della vettura. Considerando costante la velocità v del veicolo, l'assenza di slittamento e che l'intera rotazione del veicolo avvenga attorno al CIR (Centro Istantaneo di Rotazione), il modello viene descritto dal seguente sistema:

$$\begin{cases} \dot{x}(t) = v \cos(\theta) \\ \dot{y}(t) = v \sin(\theta) \\ \dot{\theta}(t) = \frac{v}{L} \tan(\delta) \end{cases} \quad (1.3)$$

I parametri fondamentali del modello sono:

- L : distanza tra asse anteriore e posteriore (passo);
- δ : angolo di sterzata (ruote anteriori);
- θ : angolo di imbardata del veicolo;
- v : velocità della macchina;
- $\dot{x}, \dot{y}$: velocità nei due assi (derivate della posizione rispetto al tempo).

Nel caso in cui si considerasse lo slittamento si passerebbe ad un modello dinamico e la velocità di rotazione non dipenderebbe più solo dalla geometria. Il modello descritto quindi è una semplificazione del modello reale, ma risulta uno strumento efficace per comprendere la cinematica di base del veicolo e la traiettoria in funzione dell'angolo di sterzo, sfruttando unicamente due assi.

1.1 Controllo veicolo

Nelle prossime sezioni verranno analizzate, dal punto di vista teorico, le tecniche di controllo utilizzate per la gestione degli attuatori presenti sul veicolo.

1.1.1 Controllo della trazione

Per il controllo della trazione, ovvero la regolazione della velocità longitudinale del veicolo, si è optato per un sistema di controllo in retroazione (*closed-loop*). Tale architettura risulta fondamentale non solo per garantire il preciso inseguimento della velocità di riferimento, ma anche per compensare i disturbi esterni non prevedibili, quali le variazioni di attrito del terreno, le pendenze o le cadute di tensione della batteria durante l'erogazione di corrente. Il sistema misura costantemente la velocità reale del veicolo e la confronta con la velocità di riferimento, generando un segnale di errore $e(t)$ da minimizzare:

$$e(t) = v_{\text{target}} - v_{\text{reale}} \quad (1.4)$$

Il feedback di velocità è fornito da un encoder incrementale calettato sull'albero motore: il sensore genera una serie di impulsi che vengono conteggiati dal microcontrollore all'interno di un periodo di campionamento a tempo fisso. Dalla derivata discreta di questi conteggi si ricava l'effettiva velocità di rotazione del rotore. Per minimizzare l'errore $e(t)$ e calcolare lo sforzo di controllo da applicare al motore, tradotto poi nel *Duty Cycle* del segnale PWM, viene impiegato un controllore PID (Proporzionale-Integrale-Derivativo). La legge di controllo nel dominio del tempo continuo è descritta dalla seguente equazione:

$$u(t) = K_p \cdot e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt} \quad (1.5)$$

dove $u(t)$ rappresenta l'azione di comando generata dal controllore, mentre K_p , K_i e K_d sono rispettivamente le costanti di guadagno dell'azione proporzionale, integrale e derivativa. Nello specifico:

- **Guadagno proporzionale (K_p):** fornisce un'uscita di controllo direttamente proporzionale all'errore istantaneo. Aumentare il guadagno K_p rende il sistema più reattivo e riduce il tempo di salita. Tuttavia, da sola non è sufficiente a garantire il perfetto inseguimento del riferimento: un'azione puramente proporzionale lascia un errore a regime e, se eccessiva, induce forti oscillazioni e instabilità.
- **Guadagno integrale (K_i):** somma l'errore nel tempo, tenendo conto della “storia” del sistema. Il suo scopo fondamentale è annullare completamente l'errore a regime. Tuttavia, un accumulo prolungato di questo termine, ad esempio durante la fase di avvio o nel caso in cui le ruote slittino senza far muovere il veicolo, porta a un eccessivo incremento dell'azione di controllo. Questo si traduce in partenze improvvise o risposte sproporzionate non appena il target cambia, problematica che nel software è stata mitigata azzerando il termine integrale in specifiche condizioni di sicurezza.
- **Guadagno derivativo (K_d):** reagisce alla velocità di variazione dell'errore, anticipandone l'andamento futuro. Funge essenzialmente da “smorzatore”, con lo scopo di contrastare le eventuali oscillazioni introdotte dalle componenti precedenti. Tuttavia, per il controllo longitudinale di veicoli di questa scala, l'azione derivativa risulta spesso superflua o persino controproducente, in quanto tende ad amplificare matematicamente le normali micro-vibrazioni meccaniche o i piccoli scatti letti dall'encoder. Per questo motivo, è prassi comune nella robotica mobile di base porre direttamente $K_d = 0$, semplificando l'architettura a un più robusto e stabile controllore PI.

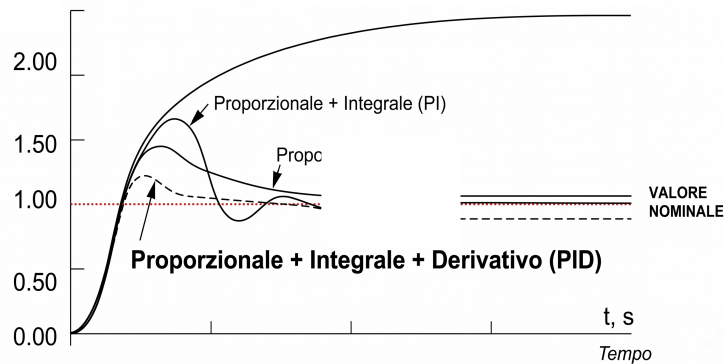


Figure 1.1

La determinazione dei parametri ottimali (K_p , K_i , K_d) prende il nome di calibrazione o *tuning*. Per la dinamica del veicolo in esame, tale procedura è stata condotta mediante calibrazione empirica in laboratorio. Partendo da valori conservativi e analizzando visivamente e tramite telemetria la risposta del motore alle variazioni di velocità richieste dal radiocomando, i guadagni sono stati ottimizzati per garantire l'inseguimento del target senza innescare instabilità.

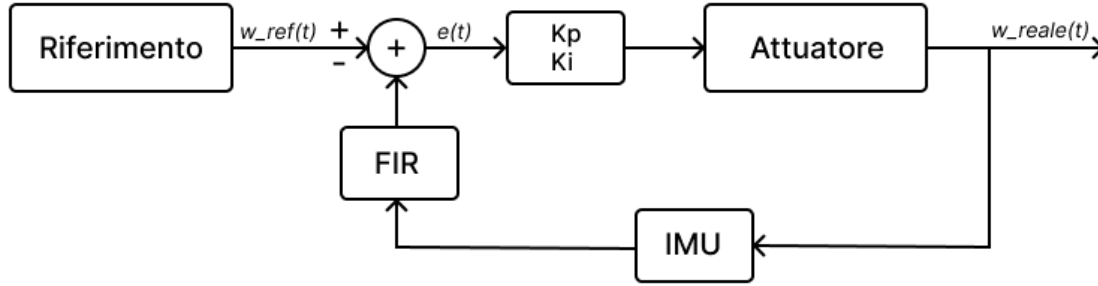


Figure 1.2

1.1.2 Controllo dello sterzo

La gestione direzionale del veicolo è affidata al sistema di sterzo delle ruote anteriori e ai segnali inviati dal radiocomando. L'attuazione meccanica è realizzata mediante un servomotore, un tipo particolare di motore che, agendo sui braccetti dello sterzo, è in grado di ruotare in una specifica posizione e mantenerla. Inoltre, in questo prototipo è stato implementato un controllo a ciclo chiuso per garantire la massima precisione nell'inseguimento della traiettoria e compensare i disturbi dinamici come gli attriti del terreno. Di conseguenza, il controllo prevede l'utilizzo di un regolatore PID, implementato all'interno del microcontrollore STM32. Il controllore riceve in ingresso l'errore $e(t)$, definito come la differenza tra l'angolo di sterzo di riferimento richiesto $\delta_{ref}(t)$, letto dal radiocomando, e l'angolo reale misurato dai sensori di bordo $\delta_{reale}(t)$:

$$e(t) = \delta_{ref}(t) - \delta_{reale}(t) \quad (1.6)$$

L'azione correttiva calcolata dall'algoritmo PID viene tradotta in un segnale di pilotaggio PWM inviato al servomotore. Per questa tipologia di attuatori, il controllo della posizione non dipende dal *Duty Cycle* percentuale classico, bensì dalla larghezza assoluta dell'impulso positivo T_{on} all'interno di un periodo fisso di 20 ms, corrispondente ad una frequenza di 50 Hz.

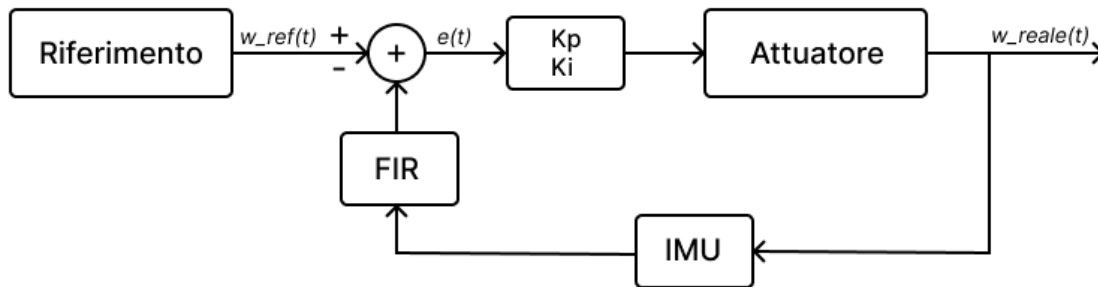


Figure 1.3

2 Hardware

Nel presente capitolo viene analizzata in dettaglio l'architettura hardware del prototipo, descrivendo le specifiche tecniche e il ruolo funzionale di ciascun componente. Verranno presi in analisi i seguenti elementi costitutivi:

- Telaio e meccanica di supporto;
- Microcontrollore Nucleo-144 STM32H755zi-q;
- Motore DC Reely 550er;
- Driver Motore Cytron MD10C R3;
- Encoder incrementale CUI Devices AMT102;
- Servomotore Miuzei MS24;
- Sistema di Alimentazione e Power Distribution Board (PDB);
- Batteria ai polimeri di litio (LiPo) 2S (2 celle);
- Unità di Inerzia Magneto-Inerziale (IMU) Bosch BNO055;
- Radiocomando e Ricevitore Microzone.

2.1 Telaio e meccaniche di supporto

La struttura portante e la cinematica del veicolo sono basate su uno chassis in scala 1:10 derivato dal settore del modellismo dinamico. Per rendere lo chassis idoneo agli scopi del progetto, la struttura originale è stata sottoposta a una serie di interventi di personalizzazione. In particolare, è stata progettata e installata una piastra di supporto superiore per incrementare la superficie utile a bordo, permettendo l'alloggiamento di tutta la componentistica elettronica.



Figure 2.1

2.2 Nucleo-144 STM32H755zi-q

L'unità di elaborazione centrale e di controllo del veicolo è costituita dalla scheda di sviluppo Nucleo-144 dotata del microcontrollore ad altissime prestazioni STM32H755ZIT6Q di STMicroelectronics. L'elemento distintivo di questo dispositivo è la sua architettura hardware *dual-core* eterogenea, che mette a disposizione due processori indipendenti a bordo dello stesso chip:

- **Core ARM Cortex-M7 (32-bit):** operante a una frequenza massima di 480 MHz.
- **Core ARM Cortex-M4 (32-bit):** operante a una frequenza massima di 240 MHz.

L'interazione e lo scambio di dati ad alta velocità tra i due core avvengono in modo sincrono tramite un'interfaccia di comunicazione interna basata su memoria condivisa e semafori hardware. La scheda offre un ricco set di periferiche hardware dedicate che risultano fondamentali per gli obiettivi del progetto: i timer avanzati per la generazione dei segnali PWM di trazione e sterzo, i timer configurati in modalità *Encoder Interface* per la lettura diretta in quadratura dell'encoder AMT102, e i bus di comunicazione analogici e digitali, come I²C o UART, per l'interfacciamento con i sensori di bordo. La configurazione delle periferiche e lo sviluppo del codice in linguaggio C sono stati interamente gestiti tramite l'ecosistema software ufficiale di STMicroelectronics, utilizzando lo strumento di generazione grafica STM32CubeMX per l'inizializzazione dell'hardware e l'ambiente di sviluppo integrato STM32CubeIDE per la scrittura, compilazione e il *debug* del *firmware*.

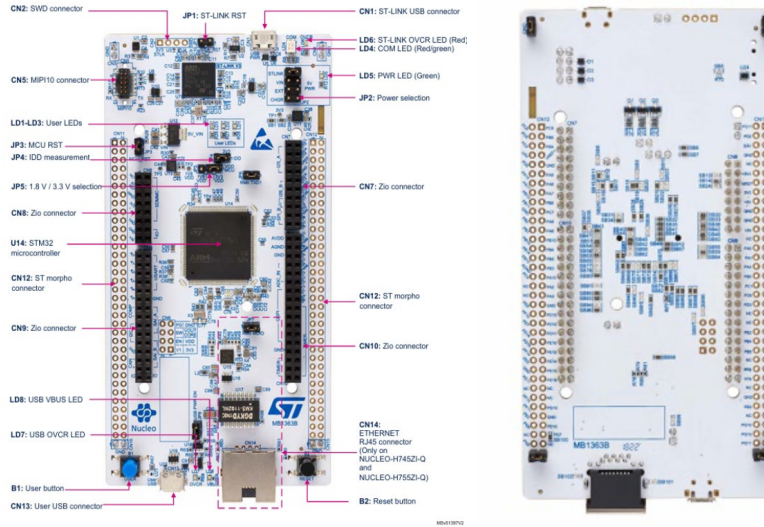


Figure 2.2

2.3 Motore DC Reely 550er

La trazione del veicolo viene garantita da un motore collegato alla trasmissione dello chassis.



Figure 2.3

In particolare, viene utilizzato un motore DC a collettore con le seguenti caratteristiche:

Parametro	Valore
Tensione nominale	4.8 – 12 VDC
Velocità di rotazione	1000 – 18000 RPM
Coppia	100 – 900 g·cm
Diametro albero	3.175 mm

2.4 Driver Motore Cytron MD10C R3

Il motore viene controllato tramite modulazione PWM attraverso il driver MD10C-R3. In particolare, il controllo del motore attraverso questo driver avviene tramite due segnali:

- **DIR:** controllo del verso di rotazione (Segnale digitale alto/basso)
- **PWM:** controllo della velocità di rotazione (segnale PWM)



Figure 2.4

2.5 Encoder incrementale CUI Devices AMT102

Inoltre, per effettuare il controllo del motore in velocità è necessario avere un feedback sulla velocità del motore stesso: ciò è stato reso possibile montando un encoder sull'asse posteriore del motore. In particolare, è stato utilizzato un encoder rotativo incrementale modello AMT102.



Figure 2.5

2.6 Servomotore Miuzei MS24

Per operare il meccanismo di sterzo del veicolo è stato impiegato un servo motore in grado di regolare il movimento angolare in modo molto preciso e proprio per questo viene comunemente utilizzato in diverse apparecchiature elettroniche e applicazioni che richiedono un controllo di posizione accurato.



Figure 2.6

Il dispositivo è controllato mediante un segnale PWM fornito in ingresso, calcolato dagli algoritmi di controllo implementati sul microcontrollore.

2.7 Alimentazione

Il sistema è composto da numerose componenti, che non operano tutte alla stessa tensione. Per questo motivo è stato necessario utilizzare dei convertitori DC-DC XL4015, così da garantire le alimentazioni necessarie.



Figure 2.7

2.7.1 Batteria ai polimeri di litio

Il sistema è alimentato da una batteria ai polimeri di litio (LiPo) 2S (2 celle) con una tensione nominale di 7.4V e una capacità di 5500 mAh, in grado di erogare una corrente di 5.5A per la durata di un'ora, garantendo una corretta alimentazione avendo una discreta autonomia.



Figure 2.8

2.8 Inertial Measurement Unit

L'IMU è un dispositivo che misura l'accelerazione, la velocità angolare e talvolta anche il campo magnetico. Solitamente integra un accelerometro, un giroscopio e un magnetometro, ognuno avente 3 assi (in questa configurazione l'IMU si dice a 9 gradi di libertà, 9DOF). Tali misure sono di fondamentale importanza per applicazioni in cui è richiesta la navigazione e il controllo di veicoli o in altri dispositivi mobili. L'IMU scelto per questa applicazione è il BNO055 prodotto dalla Bosch Sensortech. È caratterizzato dall'inclusione di un microcontrollere a 32 bit con algoritmi di sensor fusion integrati, che combinano i dati grezzi dei sensori per fornire informazioni precise su orientamento, accelerazione e rotazione dell'unità. Il BNO055 può comunicare tramite UART o I2C.

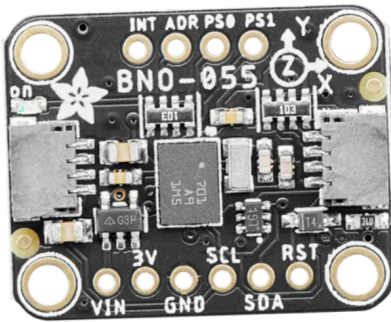


Figure 2.9

2.9 Radiocomando e Ricevitore Microzone

Il radiocomando utilizzato per inviare segnali PWM al fine di controllare la macchinina con un remote controller è il “Microzone 6CH Remote Transmitter Receiver System for Drones/Quadcopters” modello: “MC6C”.



Figure 2.10

Il radiocomando dispone di sei canali e la banda va dai 2,401 GHz a 2,479 GHz. La distanza massima di controllo è di 800m e utilizza 4 batterie xAA. Il radiocomando è inoltre dotato di ricevitore e trasmettitore che hanno le seguenti caratteristiche:

Table 2.1: *Caratteristiche del trasmettitore*

Caratteristiche	Valori
Potenza di trasmissione	≤ 70 mW
Range per il controllo	800 m
Tensione (DC)	+6 V
Peso	550 g
Modalità di modulazione	FHSS

Table 2.2: *Caratteristiche del ricevitore*

Caratteristiche	Valori
Banda di frequenza	2.400 – 2.483 GHz
Distanza lineare	> 800 m
Tensione di alimentazione (DC)	4,5 – 6 V
Tipo di segnale	PWM / SBUS
Antenna	Integrata nel ricevitore
Dimensioni	$45 \times 45 \times 10$ mm
Peso	9,6 g

2.9.1 Funzionamento

Il trasmettitore e il ricevitore comunicano tramite onde radio, le quali fungono da vettore per le istruzioni di comando generate dal pilota. Il modulo ricevitore svolge un ruolo cruciale in questa architettura: elabora i pacchetti radio in ingresso e genera in uscita un segnale PWM indipendente per ciascun canale. La variazione del *Duty Cycle* di questi segnali traduce proporzionalmente i movimenti fisici eseguiti sul radiocomando in dati quantificabili dal microcontrollore.



Figure 2.11

In fase di configurazione, la gestione della dinamica del veicolo è stata mappata in modo strategico su specifiche levette:

- **Controllo trazione (cerchio verde):** La leva di destra è dedicata alla velocità longitudinale. Il suo azionamento lungo l'asse verticale permette di regolare la potenza erogata e far avanzare o retrocedere il veicolo.
- **Controllo sterzo (cerchio giallo):** La leva di sinistra è deputata alla gestione dell'angolo di sterzata. Questa specifica scelta di *mapping* ergonomico è stata fatta perché spingere la leva orizzontalmente (a destra o a sinistra) emula in modo molto più naturale e intuitivo il movimento direzionale delle ruote. Tale spostamento trasversale si traduce direttamente nella variazione del segnale PWM inviato al servomotore.
- **Armamento motore (cerchio blu):** Lo *switch* a tre scatti, situato in alto a sinistra, funge da sistema di sicurezza (abilitazione hardware) per l'armamento del motore. Nelle posizioni estreme (completamente in alto o in basso) il motore è forzatamente spento e disarmato, mentre posizionando la levetta al centro il sistema viene armato ed è pronto a ricevere potenza dalla trazione.
- **Regolazione fine (cerchi rossi):** Infine, i *trimmer* fisici consentono di applicare un *offset* ai segnali. Questi pulsanti traslano leggermente l'intervallo di base del *Duty Cycle* sui primi quattro canali, rivelandosi essenziali per calibrare elettronicamente lo "zero" meccanico dello sterzo e del motore senza dover intervenire via software.

2.9.2 Canali del ricevitore

Il ricevitore è dotato di sei canali che permettono di gestire i segnali generati dallo spostamento delle varie levette. La seguente tabella riporta la relazione tra canali e leve.

Table 2.3: *Mappatura dei canali del radiocomando*

Canale	Leva del radiocomando
Channel 1	Leva destra orizzontale
Channel 2	Leva destra verticale
Channel 3	Leva sinistra verticale
Channel 4	Leva sinistra orizzontale
Channel 5	Levetta in alto a sinistra (a 3 posizioni)
Channel 6	Rotella in alto a destra

Per la realizzazione del progetto sono stati utilizzati solo i canali 2 e 4 per le levette.

2.10 Schema dei collegamenti

La Figura 2.12 illustra lo schema generale dei collegamenti elettrici e di segnale che costituiscono l'architettura hardware del prototipo. Il sistema è centralizzato attorno alla scheda di sviluppo STM32, la quale funge da nodo principale per l'elaborazione logica, l'acquisizione dei dati e il controllo degli attuatori.

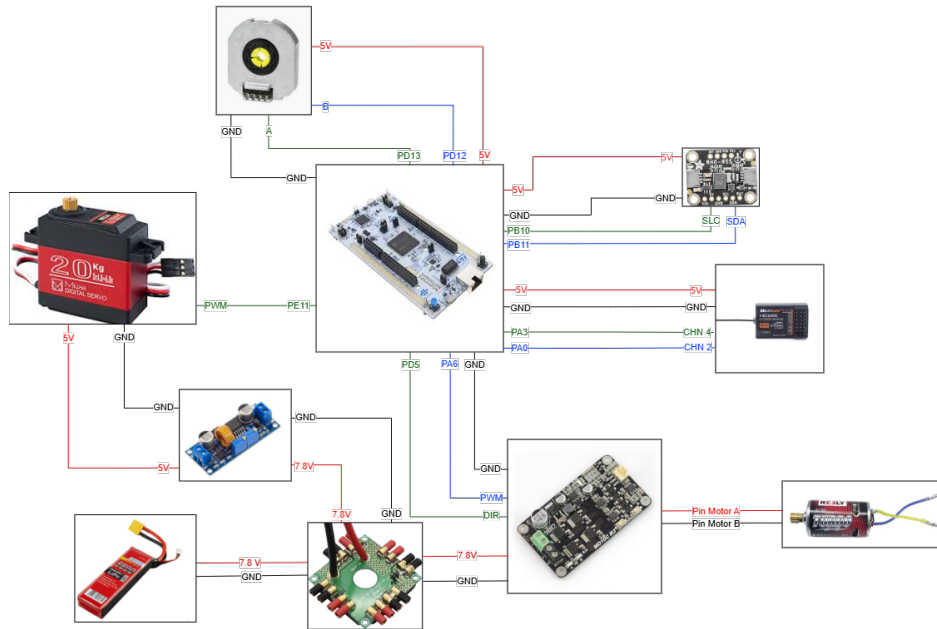


Figure 2.12: *Schema dei collegamenti*

3 Software

Il presente capitolo descrive gli applicativi software impiegati per lo sviluppo del progetto. L'utilizzo di tali strumenti ha consentito la programmazione del microcontrollore e la successiva implementazione degli algoritmi di controllo necessari al funzionamento dinamico del veicolo. Inoltre, l'infrastruttura software si è rivelata un supporto fondamentale durante la fase di *testing*, permettendo l'acquisizione e la visualizzazione in tempo reale dei parametri di telemetria e dei grafici di risposta del sistema.

3.0.1 STM32CubeIDE

Il firmware per la scheda STM32H755ZI-Q è stato interamente sviluppato in linguaggio C avvalendosi di STM32CubeIDE, l'ambiente di sviluppo integrato ufficiale di STMicroelectronics. Quest'ultimo mette a disposizione un'avanzata interfaccia grafica che, tramite un file di configurazione con estensione `.ioc`, permette di inizializzare l'hardware del microcontrollore in modo rapido e intuitivo. Tra le principali impostazioni gestite figurano:

- Configurazione dell'albero di *clock* del sistema;
- Assegnazione e configurazione dei pin di input/output (GPIO);
- Setup dei Timer hardware (es. per la generazione del PWM e la lettura dell'encoder);
- Inizializzazione delle interfacce di comunicazione (UART, I²C, SPI).

Una volta ultimata la configurazione grafica, l'IDE provvede alla generazione automatica del codice C di base, fornendo un'architettura software pre-inizializzata su cui lo sviluppatore può procedere direttamente all'implementazione della logica di controllo.

3.0.2 MATLAB

MATLAB, ambiente di calcolo numerico e linguaggio di programmazione sviluppato da MathWorks, è uno strumento ampiamente impiegato in ambito ingegneristico per l'analisi dei dati, lo sviluppo di algoritmi, la simulazione di sistemi di controllo e l'elaborazione dei segnali. Il software si distingue per una sintassi intuitiva, un'architettura ottimizzata per la gestione di matrici e vettori, e la disponibilità di numerosi *toolbox* settoriali, uniti a potenti interfacce per la visualizzazione grafica in 2D e 3D. All'interno di questo progetto, le funzionalità di *plotting* di MATLAB sono state impiegate specificamente per l'acquisizione e la visualizzazione dei dati in tempo reale (*real-time*), fornendo un prezioso supporto visivo per il monitoraggio delle variabili in gioco e la validazione del comportamento dinamico del veicolo.

3.0.3 Acquisizione dati via Terminale

Per la cattura e la verifica preliminare dei dati telemetrici inviati dal microcontrollore, ci si è avvalsi del terminale nativo di macOS in esecuzione su un MacBook Pro M2. Sfruttando la comunicazione seriale (UART via USB) tra la scheda STM32 e il computer, l'interfaccia a riga

di comando ha permesso di monitorare e registrare il flusso continuo di dati grezzi in uscita. Questa procedura si è rivelata uno step di *debug* cruciale: ha consentito di verificare l'integrità, la frequenza e la corretta formattazione dei pacchetti dati (come i conteggi dell'encoder o l'errore del PID) prima che questi venissero acquisiti ed elaborati dall'ambiente MATLAB per il *plotting* in tempo reale.

4 Implementazione del firmware

In questo capitolo verranno analizzati gli algoritmi di controllo sviluppati sul microcontrollore dal punto di vista della loro implementazione. La trattazione sarà divisa in quattro sezioni, la prima fornirà una descrizione generale della struttura del codice, mentre le successive riguarderanno ognuna un aspetto fondamentale del moto del veicolo: ossia il controllo del radiocomando, della trazione e dello sterzo.

4.1 Diagramma di flusso

Si descrivere in maniera complessiva la logica del sistema attraverso il successivo diagramma di flusso 4.1:

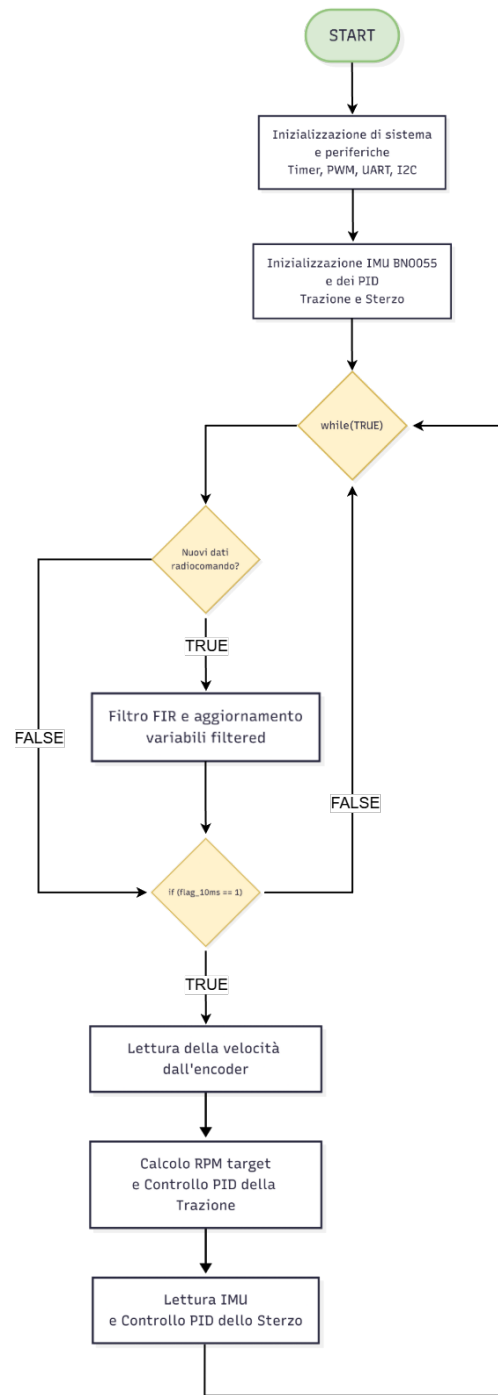


Figure 4.1: *Diagramma di flusso del codice sviluppato*

Si evidenzia come il flusso di esecuzione delle istruzioni del codice inizi con una fase preliminare di setup, in cui vengono inizializzate le periferiche di sistema (Timer, PWM, UART, I2C) e configurati i componenti essenziali, ovvero l'unità inerziale (IMU BNO055) e i due regolatori PID per il controllo della trazione e dello sterzo. Successivamente, il programma

entra in un ciclo di esecuzione infinito, rappresentato dal blocco “while(TRUE)”. All’interno di questo ciclo, il flusso operativo dipende dall’esito di due blocchi condizionali principali. Il primo verifica costantemente la ricezione di nuovi comandi impartiti tramite il radiocomando. Quando questa condizione è verificata (TRUE), i segnali grezzi in ingresso vengono elaborati attraverso un filtro FIR (Finite Impulse Response), al fine di smorzarne le oscillazioni, e vengono aggiornate le rispettive variabili filtrate che rappresentano il setpoint imposto dal pilota. Nel caso in cui non vi siano nuovi dati (FALSE), l’algoritmo salta l’elaborazione del filtro. Il flusso prosegue poi verso la seconda condizione, che rappresenta il cuore del sistema di controllo in tempo reale: la verifica della variabile temporale “flag_10ms”. Se tale variabile assume il valore 1 (TRUE), significa che è trascorso il periodo di campionamento prefissato di 10 millisecondi. In questa occorrenza, il sistema esegue in stretta sequenza le operazioni necessarie per il movimento del veicolo: procede prima con la lettura della velocità attuale dall’encoder, il calcolo degli RPM target e l’esecuzione del controllo PID della trazione; successivamente effettua la lettura del tasso di imbardata reale dall’IMU per alimentare il controllo PID dello sterzo. Al termine di queste operazioni, o nel caso in cui il periodo di 10ms non sia ancora trascorso (FALSE), il flusso ritorna all’inizio del ciclo per ripetere le verifiche. Tutti gli algoritmi utilizzati sono stati sviluppati mediante linguaggio C e l’ambiente di sviluppo STM32CubeIDE.

4.2 Implementazione relativa al radiocomando

L’implementazione relativa al radiocomando rappresenta il fulcro di questa relazione: l’obiettivo del progetto è infatti proprio quello di gestire la comunicazione tra radiocomando e microcontrollore, al fine di gestire il moto del veicolo (trazione e sterzo).

4.2.1 Impostazioni iniziali timers e calcolo preliminare di frequenza e duty

In questa sottosezione si riportano le impostazioni iniziali dei Timers e il codice utilizzato per l’acquisizione dei primi valori di frequenza e duty fondamentali per le fasi di implementazione successive. Inizialmente si è analizzato il segnale PWM restituito da ogni canale del ricevitore. Per ottenere la frequenza ed il duty cycle tutti i canali, è stato utilizzato un interrupt in modalità InputCapture che consente di studiare un segnale preso come input, generato appunto dal ricevitore. Tramite la configurazione nel file “.ioc” di questo interrupt, è necessario specificare la “Polarity Section”, ovvero a quale evento del segnale si è interessati: “Rising Edge” (fronte di salita), “Falling Edge” (fronte di discesa), “Both Edges” (fronte di salita e di discesa). Grazie a quanto appreso da relazioni precedenti in cui si era utilizzato un oscilloscopio, si è a conoscenza del fatto che tutti i segnali sono sincronizzati tra loro, ovvero hanno stessa frequenza e non sono sfasati tra loro. Questa osservazione sarà utile in un secondo momento.

Impostazioni timer

I timer permettono di misurare il tempo e quindi di ottenere i valori desiderati dalla ricevente. Per l’utilizzo dell’Input Capture (sfruttando i canali PA0 e PA3 del TIM5) si è deciso di utilizzare un timer con un clock di 64 MHz, che viene diviso dal “Prescaler”. Poiché a livello hardware il microcontrollore somma sempre 1 al valore del registro, per ottenere un

clock effettivo di 1 MHz il parametro è stato settato a 63, calcolato mediante la formula $PSC = \frac{64.000.000}{1.000.000} - 1$.

Inoltre, poiché la frequenza minima leggibile è data da $\frac{TIMCLOCK}{COUNTERPERIOD}$, si è deciso di lasciare il "Counter Period" al massimo, ovvero 0xffffffff (corrispondente a 4294967295) essendo il TIM5 un timer a 32 bit.

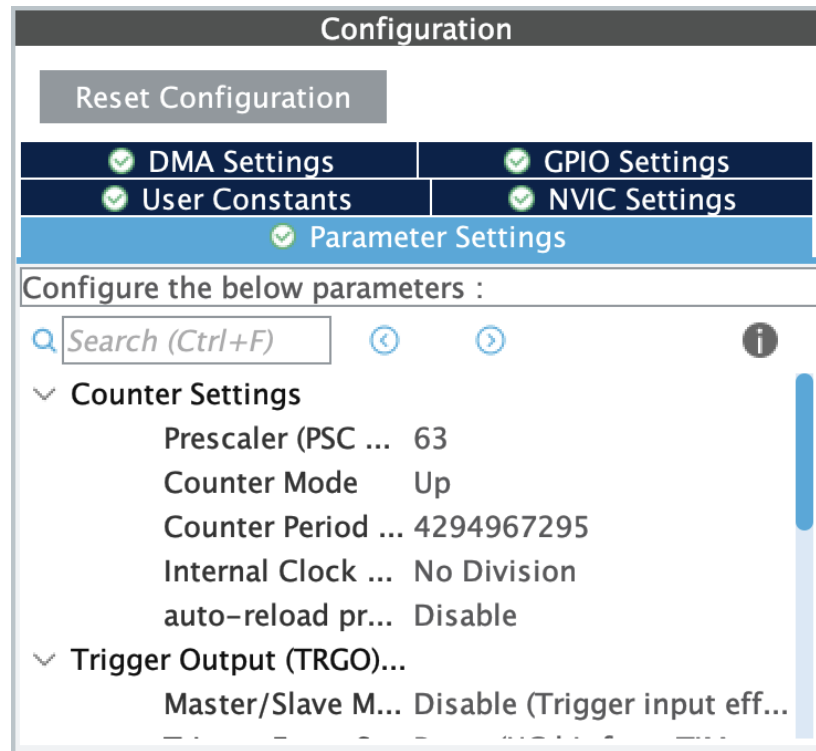


Figure 4.2: Impostazioni timer

Acquisizione del segnale PWM

Il segnale PWM in uscita dal ricevitore del radiocomando presenta una frequenza fissa (tipicamente 50 Hz, pari a un periodo di 20 ms). Pertanto, il calcolo della frequenza risulta irrilevante ai fini del controllo dinamico del veicolo. L'informazione vitale legata al comando impartito dal pilota risiede unicamente nella larghezza dell'impulso alto (T_{on}), che varia proporzionalmente al movimento delle levette in un intervallo tipicamente compreso tra 1000 μs e 2000 μs .

Per acquisire direttamente questo dato vitale, si è configurato il parametro "Polarity Selection" su "Both Edges" all'interno dell'ambiente di sviluppo, discostandosi da un approccio basato sul solo fronte di salita.

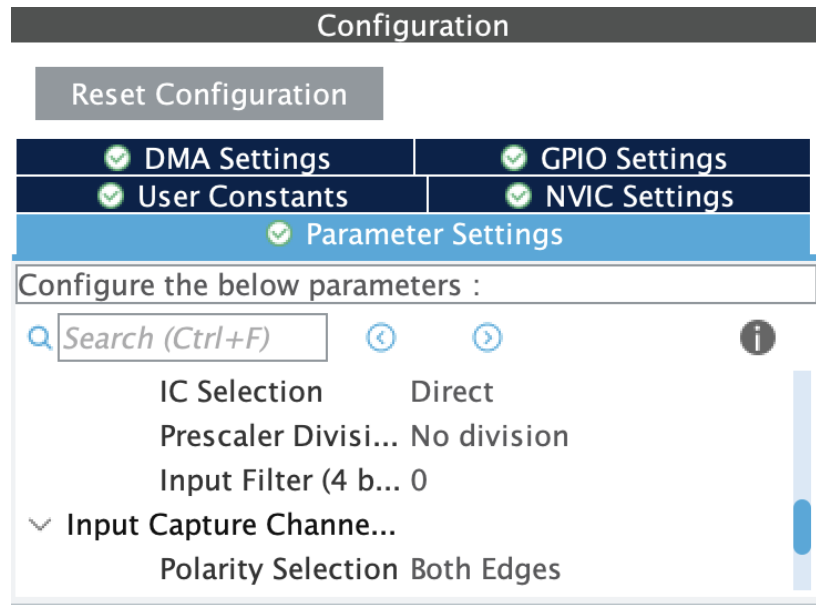


Figure 4.3: Configurazione Polarity Selection su Both Edges

La logica di acquisizione è gestita all'interno della funzione di callback `HAL_TIM_IC_CaptureCallback`. Tramite l'ausilio di flag di stato (come `Is_First_Captured_Steering`), il sistema salva il valore del contatore hardware al verificarsi del fronte di salita, commutando contestualmente la sensibilità del canale sul fronte opposto. Al successivo fronte di discesa, il contatore viene letto nuovamente per poterne calcolare la differenza temporale.

Poiché il timer opera con un clock effettivo di 1 MHz, la differenza algebrica tra le due letture restituisce la durata dell'impulso espressa direttamente in microsecondi. Tale valore viene salvato nelle variabili globali (`steering_us` e `throttle_us`) per essere impiegato come setpoint, a patto che superi il controllo di sicurezza implementato per scartare letture anomale esterne al range nominale (600–2400 μ s).

```

1 void HAL_TIM_IC_CaptureCallback(TIM_HandleTypeDef *htim)
2 {
3     // STERZO -> PA3 (TIM5 CH4)
4     if (htim->Instance == TIM5 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_4)
5     {
6         static uint32_t ic_val1_s = 0;
7         uint32_t ic_val2_s = 0;
8         uint32_t diff_steering = 0;
9
10        if (Is_First_Captured_Steering == 0)
11        {
12            ic_val1_s = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_4);
13            Is_First_Captured_Steering = 1;
14            __HAL_TIM_SET_CAPTUREPOLARITY(htim, TIM_CHANNEL_4,
15                                          TIM_INPUTCHANNELPOLARITY_FALLING);
16        }
17        else
18        {
19            ic_val2_s = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_4);

```

```

20         if (ic_val2_s > ic_val1_s)
21             diff_steering = ic_val2_s - ic_val1_s;
22         else
23             diff_steering = (0xFFFFFFFF - ic_val1_s) + ic_val2_s;
24
25         // Per lo sterzo FIR
26         if (diff_steering > 600 && diff_steering < 2400) {
27             steering_us = diff_steering;
28             new_rc_steering = 1;
29         }
30
31         Is_First_Captured_Steering = 0;
32         __HAL_TIM_SET_CAPTUREPOLARITY(htim, TIM_CHANNEL_4,
33             TIM_INPUTCHANNELPOLARITY_RISING);
34     }
35
36     // Trazione -> PA0 (TIM5 CH1)
37     if (htim->Instance == TIM5 && htim->Channel == HAL_TIM_ACTIVE_CHANNEL_1)
38     {
39         static uint32_t ic_val1_t = 0;
40         uint32_t ic_val2_t = 0;
41         uint32_t diff_throttle = 0;
42
43         if (Is_First_Captured_Throttle == 0)
44         {
45             ic_val1_t = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
46             Is_First_Captured_Throttle = 1;
47             __HAL_TIM_SET_CAPTUREPOLARITY(htim, TIM_CHANNEL_1,
48                 TIM_INPUTCHANNELPOLARITY_FALLING);
49         }
50         else
51         {
52             ic_val2_t = HAL_TIM_ReadCapturedValue(htim, TIM_CHANNEL_1);
53
54             if (ic_val2_t > ic_val1_t)
55                 diff_throttle = ic_val2_t - ic_val1_t;
56             else
57                 diff_throttle = (0xFFFFFFFF - ic_val1_t) + ic_val2_t;
58
59             if (diff_throttle > 600 && diff_throttle < 2400) {
60                 throttle_us = diff_throttle;
61                 new_rc_throttle = 1;
62             }
63
64             Is_First_Captured_Throttle = 0;
65             __HAL_TIM_SET_CAPTUREPOLARITY(htim, TIM_CHANNEL_1,
66                 TIM_INPUTCHANNELPOLARITY_RISING);
67         }
68     }
69 }

```

Listing 4.1: "Codice per la lettura del segnale PWM tramite Input Capture"

Gestione degli Interrupt e ottimizzazione dei Timer

A differenza delle classiche implementazioni hardware che prevedono l'azzeramento forzato del contatore del timer ad ogni fronte di salita (metodologia che impedisce la lettura simultanea di più canali e costringe all'utilizzo di molteplici timer), in questo progetto si è optato per un approccio software non distruttivo.

Come si evince dalla configurazione di sistema in STM32CubeMX (Figura 4.4), il timer dedicato alla decodifica (TIM5) è attivamente impiegato sui canali 1 e 4 in modalità *Input Capture direct mode*, ma il suo contatore viene fatto avanzare in modo continuo (fino al suo limite massimo a 32 bit, pari a 4294967295) senza mai subire reset forzati. All'attivazione dell'interrupt, la *callback* si limita a "fotografare" il valore assoluto del registro (tramite la funzione `HAL_TIM_ReadCapturedValue`).

The screenshot shows the STM32CubeMX Pinout & Configuration window. The 'Timers' table on the left shows TIM5 selected for Cortex-M7. The 'TIM5 Mode and Configuration' panel on the right shows the following settings:

Parameter	Value
Mode	Input Capture direct mode
Trigger Source	Disable
Clock Source	Disable
Channel1	Input Capture direct mode
Channel2	Disable
Channel3	Disable
Channel4	Input Capture direct mode
Combined Channels	Disable
ETR IO as Clearing Source	Disable
XOR activation	Disable
One Pulse Mode	Disable

The 'Configuration' panel shows the following settings:

Parameter	Value
Reset Configuration	Reset
DMA Settings	Enabled
GPIO Settings	Enabled
User Constants	Enabled
NVIC Settings	Enabled
Parameter Settings	Enabled

The 'Counter Settings' panel shows the following settings:

Parameter	Value
Prescaler (PSC ...)	63
Counter Mode	Up
Counter Period ...	4294967295
Internal Clock ...	No Division
auto-reload pr...	Disable

The 'Trigger Output (TRGO)...' panel shows the following settings:

Parameter	Value
Master/Slave M...	Disable (Trigger input eff...)

Figure 4.4: Configurazione del TIM5 con i canali 1 e 4 attivi

La durata dell'impulso viene così ricavata puramente per differenza matematica tra il timestamp di discesa e quello di salita. Per prevenire errori di calcolo nel raro caso in cui avvenga un overflow del timer proprio durante un impulso, l'algoritmo include un controllo condizionale che compensa il riavvio del conteggio sommando il complemento a due del limite massimo.

Questa architettura software garantisce numerosi vantaggi:

- **Ottimizzazione delle risorse:** consente di gestire in modo totalmente asincrono e indipendente l'acquisizione dei comandi Xa e Xb utilizzando un unico timer (TIM5) e due soli canali, riducendo il carico computazionale e liberando le restanti periferiche del microcontrollore.
- **Assenza di interferenze:** poiché il contatore principale non viene mai azzerato, l'arrivo di un interrupt sul canale dedicato a Xb non corrompe in alcun modo il conteggio temporale in corso sul canale di Xa, garantendo una precisione assoluta su entrambi i segnali in ingresso.
- **Flessibilità della polarità:** la commutazione dinamica della polarità (da *Falling* a *Rising* e viceversa) viene gestita direttamente all'interno della singola routine di interrupt.

Struttura codice principale

In questo progetto si è scelto di centralizzare l'acquisizione e il filtraggio dei comandi direttamente all'interno del file `main.c`. Questa architettura riduce l'overhead delle chiamate a funzioni, ottimizzando i tempi di esecuzione per il sistema di controllo in tempo reale. Il codice si articola in tre concetti fondamentali per il trattamento dei dati:

1. Struttura dati centralizzata

Invece di utilizzare strutture separate unicamente per la lettura temporale dei segnali, lo stato globale e le cinematiche della vettura sono state incapsulate in un'unica `struct` denominata `vehicleData`. Questa struttura funge da raccoglitore univoco per tutte le metriche fondamentali, semplificando enormemente la gestione della memoria e il passaggio di parametri alle funzioni di controllo.

```
1 typedef struct VehicleData {
2     // Trazione (Xa)
3     int counts;
4     int ref_count;
5     int delta_count;
6     float motor_speed_RPM;
7     float linear_speed_m_s;
8     float motor_speed_ref_RPM;
9     uint8_t motor_direction_ref;
10
11     // Sterzo (Xb)
12     double yaw_rate_rad_sec;
13     double yaw_rate_deg_sec;
14     double yaw_rate_ref_rad_sec;
15 } vehicleData;
```

Listing 4.2: "Definizione della struttura dati del veicolo"

2. Stabilizzazione e Zona Morta (Dead Band)

I potenziometri fisici presenti sui radiocomandi commerciali sono intrinsecamente soggetti a lievi fluttuazioni elettriche e rumore meccanico. Se non filtrati, tali disturbi farebbero percepire al microcontrollore un comando variabile anche quando le levette sono a riposo (centrate idealmente a 1500 μ s), causando impercettibili movimenti dei motori. Per ovviare a questo problema, è stata implementata via software una "zona morta" di sicurezza. Come visibile nel codice, qualsiasi variazione del segnale di trazione (**Xa**) e di sterzo (**Xb**) che ricada nell'intorno del punto di neutro viene forzatamente ignorata, garantendo il perfetto arresto del veicolo e l'allineamento delle ruote anteriori.

```
1 // Zona morta per la trazione (Xa)
2 if(throttle_filtered > 1560.0f) {
3     target_dir = 1;
4     target_rpm = ((throttle_filtered - 1560.0f) * MAX_TARGET_RPM) /
5         433.0f;
6 }
7 else if(throttle_filtered < 1440.0f) {
8     target_dir = 0;
9     target_rpm = ((1440.0f - throttle_filtered) * MAX_TARGET_RPM) /
10         449.0f;
11 }
12 else {
13     target_rpm = 0.0f;
14     target_dir = 0;
15     pid_traction.lterm = 0;
16     pid_traction.e_old = 0;
17 }
18 // Zona morta per lo sterzo (Xb)
19 if (steering_filtered > 1460.0f && steering_filtered < 1560.0f) {
20     pid_steering.lterm = 0;
21     pid_steering.e_old = 0;
22     u_sterzo = 0.0f;
23 }
```

Listing 4.3: "Implementazione della zona morta per Xa e Xb"

3. Ciclo Infinito (Main Loop)

Tutte queste operazioni avvengono costantemente all'interno del ciclo infinito `while(1)`. Qui i segnali grezzi intercettati dagli interrupt vengono passati prima attraverso un filtro FIR per regolarizzarne l'andamento, poi limitati dalle zone morte descritte in precedenza e infine mappati nei riferimenti (setpoint) da fornire agli anelli di controllo PID.

Operazioni di inizializzazione e avvio dei Timer

L'approccio implementato per l'avvio e la configurazione del sistema all'interno del `main.c` si basa su una logica del tutto asincrona e non bloccante. Le operazioni iniziali si limitano ad avviare le periferiche necessarie in sequenza, delegando l'intera logica di allineamento al fronte d'onda alla macchina a stati finiti (gestita tramite i flag software) implementata direttamente all'interno della routine di *interrupt*. In particolare, per quanto riguarda l'acquisizione dei segnali provenienti dal radiocomando (Xa per la trazione e Xb per lo sterzo), è sufficiente

abilitare gli *interrupt* legati all'Input Capture sui canali 1 e 4 del TIM5. Contestualmente, viene avviato il timer di base (TIM6) che scandisce il ciclo di controllo primario ogni 10 ms.

```

1 // Avvio del Timer per il ciclo di esecuzione a 10ms
2 HAL_TIM_Base_Start_IT(&htim6);
3 // Avvio degli interrupt di Input Capture per Xa (Trazione) e Xb (Sterzo)
4 HAL_TIM_IC_Start_IT(&htim5, TIM_CHANNEL_1);
5 HAL_TIM_IC_Start_IT(&htim5, TIM_CHANNEL_4);
6 // Inizializzazione delle strutture PID per trazione e sterzo
7 init_PID(&pid_traction, TRACTION_SAMPLING_TIME, MAX_U_TRACTION,
8         MIN_U_TRACTION);
9 tune_PID(&pid_traction, KP_TRACTION, KI_TRACTION, 0);
10 init_PID(&pid_steering, STEERING_SAMPLING_TIME, MAX_U_STEERING,
11         MIN_U_STEERING);
12 tune_PID(&pid_steering, KP_STEERING, KI_STEERING, 0);

```

Listing 4.4: "Avvio delle periferiche hardware e del sistema di controllo"

Questo paradigma architetturale snellisce il codice sorgente e riduce i tempi di accensione del sistema (*boot*), scongiurando inoltre il rischio di blocchi critici (*deadlock*) nel caso in cui il radiocomando sia spento al momento dell'accensione del microcontrollore. Essendo privo di attese bloccanti, il flusso principale continua la sua esecuzione naturale, inizializzando i sensori (come l'IMU via I²C) e posizionando le variabili di sicurezza nelle relative zone morte. Ciò garantisce che il veicolo venga mantenuto in uno stato stazionario sicuro fino all'effettiva ricezione del primo pacchetto dati valido.

Ciclo di esecuzione principale (Main Loop)

Una volta concluse le operazioni di inizializzazione e avviate le periferiche, il firmware entra nel ciclo di esecuzione infinito `while(1)`. La struttura di questo ciclo è stata progettata per separare logicamente l'acquisizione asincrona dei comandi radio dall'anello di controllo deterministico del veicolo. Il ciclo si divide in due macro-blocchi fondamentali:

1. **Acquisizione e validazione comandi:** non appena i flag `new_rc_throttle` e `new_rc_steering` segnalano la disponibilità di un nuovo dato temporale (Xa e Xb), il sistema provvede a validare e pre-filtrare i segnali grezzi, salvandoli nelle variabili globali di stato.
2. **Anello di controllo (10 ms):** sfruttando un flag hardware (`Flag_10ms`) generato dall'interrupt del TIM6, il sistema esegue il calcolo delle leggi di controllo a una frequenza fissa e predicibile (100 Hz). In questa fase vengono acquisiti i feedback dai sensori (Encoder e IMU), vengono generati i setpoint e calcolate le azioni di controllo PID.

Di seguito è riportata la struttura architetturale del ciclo principale:

```

1 while (1)
2 {
3     // --- 1. ACQUISIZIONE E VALIDAZIONE COMANDI RADIO ---
4     if(new_rc_throttle)
5     {
6         new_rc_throttle = 0;
7         // ... (Filtro e validazione segnale Xa) ...
8     }
9
10    if(new_rc_steering)

```

```

11  {
12      new_rc_steering = 0;
13      // ... (Filtro e validazione segnale Xb) ...
14  }
15
16  // --- 2. ANELLO DI CONTROLLO DETERMINISTICO (10 ms) ---
17  if (Flag_10ms == 1)
18  {
19      Flag_10ms = 0;
20
21      // Acquisizione feedback sensori (Encoder, BN0055)
22      // ...
23
24      // Mappatura comandi Xa e Xb in setpoint fisici (RPM e rad/s)
25      // ...
26
27      // Esecuzione PID Trazione e Sterzo
28      // ...
29
30      // Applicazione output ai motori e telemetria
31      // ...
32  }
33 }

```

Listing 4.5: "Struttura a macro-blocchi del loop infinito principale"

L'implementazione matematica dei filtri di validazione e le logiche di mappatura dei segnali Xa e Xb nei rispettivi setpoint fisici verranno analizzate in dettaglio nei successivi capitoli, dedicati al **Controllo della Trazione** e al **Controllo dello Sterzo**.

4.3 Gestione singoli componenti

Dedicare un paragrafo ad ogni componente in cui viene spiegato il codice che lo gestisce. Inserite prima una descrizione generale di come si gestisce un componente di quel tipo (aiutandovi con schemi o altro). Poi le configurazioni che sono state fatte sul .ioc per poterlo utilizzare (quale periferiche sono state abilitate, come sono state configurate.. mettete degli screen del .ioc). Infine inserite e spiegate le parti di codice che lo gestiscono.

NOTA per il codice: per rendere il codice modulare e riutilizzabile è buona prassi non caricare troppo il main ma creare una libreria per ogni componente. Quindi il componente1 avrà un header file "componente1.h" e un source file "componente1.c", nel source file sono implementate le funzioni che gestiscono il componente, nell'header file ci sono i prototipi di tali funzioni in modo che esse possano essere usate nel main includendolo.

NOTA per il codice: non inserire mai nel codice dei parametri numerici senza contesto ma renderli delle costanti definite (usando la direttiva %define). Se sono delle costanti relative ad uno specifico componente vanno inserite nel relativo header file.

NOTA per il codice: per inserire il codice, anziché utilizzare screenshot utilizzate i seguenti comandi.

Per codice scritto in latex (con linguaggio Python):

Listing 4.6: "Codice in Python"

```
import numpy as np

def incmatrix(genl1, genl2):
    m = len(genl1) # The length of the first array
    n = len(genl2) # The length of the second array
    sum = 0

    # Compute
    for i in range(0, n):
        for j in range(0, m):
            sum += genl1[n]*genl2[m]

    # Print
    print("The sum is %2d" %(sum))

    return M
```

Per codice richiamato da file (con linguaggio C):

```
1 void main(int n1){
2     int a = 1;           // The first number
3     int b = 2;           // The second number
4     return a+b*n1;      // The sum of the numbers
5 }
```

Listing 4.7: "Codice in C"

Funziona anche per codice MATLAB:

```
1 %% PREPARE WORKSPACE
2 close all
3 clearvars
4 clc
5
6 %% OPERATIONS
7 sayhello;
8
9 %% FUNCTIONS
10 function sayhello
11     fprintf("Hello world!");
12 end
```

Listing 4.8: "Codice in MATLAB"

4.3.1 Gestione componente1

4.3.2 Gestione componente2

4.4 Funzionamento complessivo

Dopo aver spiegato il codice che gestisce i singoli componenti, inserire e commentare le porzioni di codice relative al funzionamento complessivo del programma.

5 Test e risultati

Dedicate un capitolo a tutti i test effettuati con relativi risultati, includete sia i test finali relativi al funzionamento complessivo, sia i test delle singole parti (se significativi).

Descrivete in dettaglio le condizioni in cui sono stati svolti i test in modo che siano ripetibili.

Durante i test fate dei video e acquisite i dati (es usando la funzione di log di Putty), per i test più rilevanti è opportuno consegnare anche questo materiale per documentare i test effettuati.

Nella relazione riportate i risultati con dei grafici come quello in Figura 5.1 (inserite sempre nei grafici le label sugli assi con grandezza e relativa unità di misura). Commentate in modo critico i risultati ottenuti, evidenziando sia quelli positivi sia quelli negativi.

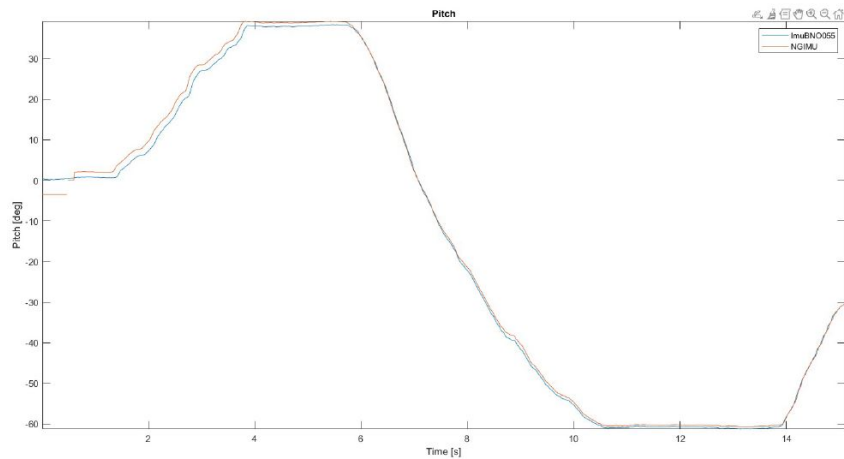


Figure 5.1: *Esempio di grafico*

5.1 Test1

Per ogni test riportate: cosa state testando, in che condizioni si è svolto il test, riferimenti a video/file di log che consegnate insieme a relazione e codice, grafici dei risultati, commento dei risultati.

5.2 Test2

Conclusioni e sviluppi futuri

Riassumere brevemente il lavoro svolto rispetto al task assegnato, evidenziando quali risultati sono stati raggiunti e quali no. Se ci sono aspetti del task non completati spiegare quali sono stati i problemi riscontrati in merito.

Inserire considerazioni personali su possibili sviluppi futuri dell'attività svolta (idee per migliorarla che non avete avuto modo di sperimentare, aspetti che suggerite di approfondire, problemi da risolvere..).

A Appendice1

Se necessario ricorrete alle appendici per spiegare le parti "di contorno" dell'attività svolta e/o ciò che non riuscite ad inserire nello schema generale dei capitoli della relazione (es acquisizione dei dati con Matlab).

Bibliografia